

Introduction to Python Programming

Chapter 17: Classes - Advanced

Introduction

You've learned how to create classes, add methods, and use special methods like `__str__` and `__repr__`. Now it's time to explore advanced class concepts that give you even more control and flexibility. This chapter covers class variables vs instance variables, static methods, class methods, the Singleton pattern, enumerations, and nested classes. These are powerful tools that professional Python developers use every day.

1. Class Variables vs Instance Variables

So far, you've only worked with **instance variables** - variables that belong to each individual object. But classes can also have **class variables** - variables shared by ALL instances of the class.

Instance Variables (Review)

Each object gets its own copy:

```
class Dog:
    def __init__(self, name):
        self.name = name # Instance variable

dog1 = Dog("Buddy")
dog2 = Dog("Max")

print(dog1.name) # Buddy
print(dog2.name) # Max
```

`dog1` and `dog2` each have their own `name` variable.

Class Variables: Shared by All Instances

Class variables are defined directly in the class, outside any method:

```

class Dog:
    species = "Canis familiaris" # Class variable

    def __init__(self, name):
        self.name = name # Instance variable

dog1 = Dog("Buddy")
dog2 = Dog("Max")

print(dog1.species) # Canis familiaris
print(dog2.species) # Canis familiaris
print(Dog.species) # Canis familiaris

```

Key observations:

- `species` is the same for ALL dogs
- You can access it through any instance OR through the class itself
- Changing it affects all instances

Practical Example: Counting Instances

```

class Student:
    total_students = 0 # Class variable

    def __init__(self, name):
        self.name = name # Instance variable
        Student.total_students += 1

    @classmethod
    def get_count(cls):
        return cls.total_students

# Create students
student1 = Student("Alice")
print(f"Total students: {Student.total_students}") # 1

student2 = Student("Bob")
print(f"Total students: {Student.total_students}") # 2

student3 = Student("Charlie")
print(f"Total students: {Student.get_count()}") # 3

```

When to Use Each

Instance variables (with `self`):

- Data unique to each object
- Name, age, id, balance, etc.
- Different for every instance

Class variables (defined in class body):

- Data shared by all instances
- Constants, counters, default values
- Same for every instance

Modifying Class Variables

```
class BankAccount:
    interest_rate = 0.02 # Class variable (2%)

    def __init__(self, balance):
        self.balance = balance

    def add_interest(self):
        self.balance += self.balance * BankAccount.interest_rate

# All accounts use the same rate
account1 = BankAccount(1000)
account2 = BankAccount(2000)

account1.add_interest() # Uses 2% rate
print(account1.balance) # 1020.0

# Change rate for ALL accounts
BankAccount.interest_rate = 0.03

account2.add_interest() # Uses new 3% rate
print(account2.balance) # 2060.0
```

Warning: If you assign to a class variable through an instance, you create a NEW instance variable instead:

```
class Example:
    shared = 10

obj1 = Example()
obj2 = Example()

print(obj1.shared) # 10
print(obj2.shared) # 10

obj1.shared = 20 # Creates NEW instance variable!
```

```
print(obj1.shared) # 20 (instance variable)
print(obj2.shared) # 10 (still using class variable)
print(Example.shared) # 10 (class variable unchanged)
```

2. Static Methods

Static methods are functions that belong to a class but don't access instance or class variables. They're utility functions related to the class.

Creating Static Methods

Use the `@staticmethod` decorator:

```
class MathHelper:
    @staticmethod
    def add(a, b):
        return a + b

    @staticmethod
    def multiply(a, b):
        return a * b

# Call without creating an instance
result = MathHelper.add(5, 3)
print(result) # 8

result = MathHelper.multiply(4, 7)
print(result) # 28
```

Key characteristics:

- No `self` or `cls` parameter
- Can't access instance or class variables
- Called through the class (or through instances, but why would you?)
- Just regular functions that live in the class namespace

When to Use Static Methods

Use static methods for utility functions that:

- Are related to the class conceptually
- Don't need access to instance or class data
- Help organize code

```

class Temperature:
    @staticmethod
    def celsius_to_fahrenheit(celsius):
        return (celsius * 9/5) + 32

    @staticmethod
    def fahrenheit_to_celsius(fahrenheit):
        return (fahrenheit - 32) * 5/9

# Use without creating objects
print(Temperature.celsius_to_fahrenheit(25)) # 77.0
print(Temperature.fahrenheit_to_celsius(77)) # 25.0

```

Practical Example: Validation

```

class User:
    def __init__(self, username, email):
        if not User.is_valid_email(email):
            raise ValueError("Invalid email")
        self.username = username
        self.email = email

    @staticmethod
    def is_valid_email(email):
        return "@" in email and "." in email

# Use the static method
print(User.is_valid_email("test@example.com")) # True
print(User.is_valid_email("invalid")) # False

# Create user (validation happens automatically)
user = User("alice", "alice@email.com")

```

3. Class Methods

Class methods receive the class itself as the first argument (not an instance). They can access and modify class variables.

Creating Class Methods

Use the `@classmethod` decorator:

```

class Pizza:
    def __init__(self, ingredients):
        self.ingredients = ingredients

```

```

@classmethod
def margherita(cls):
    return cls(["mozzarella", "tomatoes", "basil"])

@classmethod
def pepperoni(cls):
    return cls(["mozzarella", "tomatoes", "pepperoni"])

def __str__(self):
    return f"Pizza with {' '.join(self.ingredients)}"

# Create pizzas using class methods
pizza1 = Pizza.margherita()
pizza2 = Pizza.pepperoni()

print(pizza1) # Pizza with mozzarella, tomatoes, basil
print(pizza2) # Pizza with mozzarella, tomatoes, pepperoni

```

Key characteristics:

- First parameter is `cls` (the class itself)
- Can access and modify class variables
- Often used as alternative constructors
- Can create and return instances

Alternative Constructors

Class methods are perfect for creating objects in different ways:

```

class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    @classmethod
    def from_string(cls, date_string):
        year, month, day = map(int, date_string.split('-'))
        return cls(year, month, day)

    @classmethod
    def today(cls):
        import datetime
        today = datetime.date.today()
        return cls(today.year, today.month, today.day)

```

```

def __str__(self):
    return f"{self.year}-{self.month:02d}-{self.day:02d}"

# Different ways to create dates
date1 = Date(2024, 12, 25)
date2 = Date.from_string("2024-12-25")
date3 = Date.today()

print(date1)
print(date2)
print(date3)

```

Accessing Class Variables

```

class Employee:
    raise_amount = 1.04 # Class variable

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def apply_raise(self):
        self.salary = int(self.salary * self.raise_amount)

    @classmethod
    def set_raise_amount(cls, amount):
        cls.raise_amount = amount

emp1 = Employee("Alice", 50000)
emp2 = Employee("Bob", 60000)

# Change raise amount for all employees
Employee.set_raise_amount(1.05)

emp1.apply_raise()
print(emp1.salary) # 52500 (5% raise)

```

Static Method vs Class Method

When to use each:

```

class MyClass:
    class_variable = 10

    @staticmethod
    def static_method(x):
        # Can't access class_variable

```

```

        return x * 2

    @classmethod
    def class_method(cls, x):
        # CAN access class_variable
        return x * cls.class_variable

print(MyClass.static_method(5)) # 10
print(MyClass.class_method(5)) # 50

```

Use static methods when:

- Function doesn't need class or instance data
- Pure utility function

Use class methods when:

- Need to access or modify class variables
- Creating alternative constructors
- Working with the class itself

4. Enumerations (Enums)

Enumerations are a way to define a set of named constants. They make code more readable and prevent invalid values.

Creating Enums

```

from enum import Enum

class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3

# Use enum values
favorite = Color.RED
print(favorite)          # Color.RED
print(favorite.name)     # RED
print(favorite.value)    # 1

```

Why Use Enums?

Without enums (bad):


```
# Using strings - error prone!
status = "pending"
# Later...
if status == "Pending": # Oops, capital P - won't match!
    print("Still waiting")
```

With enums (good):

```
from enum import Enum

class Status(Enum):
    PENDING = 1
    APPROVED = 2
    REJECTED = 3

order_status = Status.PENDING

if order_status == Status.PENDING:
    print("Still waiting") # Works perfectly!
```

Practical Example: Days of the Week

```
from enum import Enum

class Day(Enum):
    MONDAY = 1
    TUESDAY = 2
    WEDNESDAY = 3
    THURSDAY = 4
    FRIDAY = 5
    SATURDAY = 6
    SUNDAY = 7

def is_weekend(day):
    return day in (Day.SATURDAY, Day.SUNDAY)

today = Day.FRIDAY
print(f"Is {today.name} a weekend? {is_weekend(today)}")
```

Auto-Numbering Enums

```
from enum import Enum, auto

class Status(Enum):
    PENDING = auto()    # 1
    APPROVED = auto()   # 2
    REJECTED = auto()   # 3

print(Status.PENDING.value)    # 1
print(Status.APPROVED.value)   # 2
```

Iterating Through Enums

```
from enum import Enum

class Size(Enum):
    SMALL = "S"
    MEDIUM = "M"
    LARGE = "L"
    XLARGE = "XL"

# Print all sizes
for size in Size:
    print(f"{size.name}: {size.value}")
```

Enums in Classes

```
from enum import Enum

class Order:
    class Status(Enum):
        PENDING = 1
        SHIPPED = 2
        DELIVERED = 3

    def __init__(self, order_id):
        self.order_id = order_id
        self.status = Order.Status.PENDING

    def ship(self):
        if self.status == Order.Status.PENDING:
            self.status = Order.Status.SHIPPED

    def deliver(self):
        if self.status == Order.Status.SHIPPED:
            self.status = Order.Status.DELIVERED
```

```
def __str__(self):
    return f"Order {self.order_id}: {self.status.name}"

order = Order("12345")
print(order)  # Order 12345: PENDING
order.ship()
print(order)  # Order 12345: SHIPPED
```

5. The Singleton Pattern

The **Singleton pattern** ensures only ONE instance of a class exists. When you try to create another, it returns the existing one.

The Problem

Sometimes you want to ensure only one instance exists:

- Database connections
- Configuration managers
- Logging systems

```
# Normal class - creates multiple instances
class Database:
    def __init__(self):
        print("Connecting to database...")

db1 = Database()  # Connecting to database...
db2 = Database()  # Connecting to database... (unnecessary!)
```

Implementing Singleton

```
class Singleton:
    _instance = None  # Class variable to store the instance

    def __new__(cls):
        if cls._instance is None:
            print("Creating new instance")
            cls._instance = super().__new__(cls)
        else:
            print("Returning existing instance")
            return cls._instance

# Try to create multiple instances
obj1 = Singleton()  # Creating new instance
obj2 = Singleton()  # Returning existing instance
```

```
obj3 = Singleton() # Returning existing instance

print(obj1 is obj2) # True - same object!
print(obj2 is obj3) # True - same object!
```

Understanding `__new__`

`__new__` VS `__init__`:

- `__new__` creates the object (runs first)
- `__init__` initializes the object (runs second)

```
class Example:
    def __new__(cls):
        print("__new__ called")
        return super().__new__(cls)

    def __init__(self):
        print("__init__ called")

obj = Example()
# Output:
# __new__ called
# __init__ called
```

For Singleton, we control `__new__` to return the same object every time.

Practical Singleton: Configuration Manager

```
class Config:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.settings = {}
        return cls._instance

    def set(self, key, value):
        self.settings[key] = value

    def get(self, key):
        return self.settings.get(key)

# Use from different parts of the program
config1 = Config()
```

```
config1.set("theme", "dark")

config2 = Config() # Gets the same instance
print(config2.get("theme")) # dark

print(config1 is config2) # True
```

Thread-Safe Singleton (Advanced)

For programs using threading, you need to prevent race conditions:

```
import threading

class ThreadSafeSingleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance
```

When to Use Singleton

Use Singleton for:

- Database connection managers
- Configuration settings
- Logging systems
- Resource managers (one printer, one cache, etc.)

Don't use Singleton for:

- Regular data objects
- When you might need multiple instances later
- Testing becomes harder (can't easily create fresh instances)

6. Nested Classes

A **nested class** is a class defined inside another class. It's useful for helper classes that are only used within the outer class.

Basic Nested Class

```
class University:
    class Student:
        def __init__(self, name, student_id):
            self.name = name
            self.student_id = student_id

        def __str__(self):
            return f"Student: {self.name} (ID: {self.student_id})"

    def __init__(self, name):
        self.name = name
        self.students = []

    def add_student(self, name, student_id):
        student = University.Student(name, student_id)
        self.students.append(student)

# Create university and add students
uni = University("Tech University")
uni.add_student("Alice", "001")
uni.add_student("Bob", "002")

for student in uni.students:
    print(student)
```

Why Use Nested Classes?

Organization: Groups related classes together

```
class Car:
    class Engine:
        def __init__(self, horsepower):
            self.horsepower = horsepower

        def start(self):
            print(f"Engine starting ({self.horsepower} HP)")

    def __init__(self, make, model, horsepower):
        self.make = make
        self.model = model
        self.engine = Car.Engine(horsepower)

    def start(self):
        print(f"Starting {self.make} {self.model}")
        self.engine.start()
```

```
car = Car("Toyota", "Camry", 200)
car.start()
# Starting Toyota Camry
# Engine starting (200 HP)
```

Practical Example: Linked List Node

```
class LinkedList:
    class Node:
        def __init__(self, data):
            self.data = data
            self.next = None

    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = LinkedList.Node(data)

        if not self.head:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

l1 = LinkedList()
l1.append(1)
l1.append(2)
l1.append(3)
l1.display() # 1 -> 2 -> 3 -> None
```

Accessing Nested Classes

```
class Outer:
    class Inner:
```

```

    def inner_method(self):
        print("Inner method")

    def outer_method(self):
        print("Outer method")

# Create inner class instance
inner = Outer.Inner()
inner.inner_method()

# Or through outer instance
outer = Outer()
inner2 = Outer.Inner()
inner2.inner_method()

```

Nested Class with Enum

```

from enum import Enum

class Restaurant:
    class MenuCategory(Enum):
        APPETIZER = 1
        MAIN_COURSE = 2
        DESSERT = 3
        BEVERAGE = 4

    class MenuItem:
        def __init__(self, name, price, category):
            self.name = name
            self.price = price
            self.category = category

        def __str__(self):
            return f"{self.name} (${self.price}) - {self.category.name}"

    def __init__(self, name):
        self.name = name
        self.menu = []

    def add_item(self, name, price, category):
        item = Restaurant.MenuItem(name, price, category)
        self.menu.append(item)

restaurant = Restaurant("Italian Bistro")
restaurant.add_item("Caesar Salad", 8.99, Restaurant.MenuCategory.APPETIZER)
restaurant.add_item("Pasta Carbonara", 15.99, Restaurant.MenuCategory.MAIN_COURSE)

for item in restaurant.menu:

```



```
print(item)
```

7. Putting It All Together: Advanced Example

Let's combine multiple concepts:

```
from enum import Enum, auto

class Library:
    class BookStatus(Enum):
        AVAILABLE = auto()
        CHECKED_OUT = auto()
        RESERVED = auto()

    class Book:
        _book_count = 0 # Class variable

        def __init__(self, title, author, isbn):
            self.title = title
            self.author = author
            self.isbn = isbn
            self.status = Library.BookStatus.AVAILABLE
            Library.Book._book_count += 1

        @classmethod
        def get_book_count(cls):
            return cls._book_count

        @staticmethod
        def validate_isbn(isbn):
            return len(isbn) == 13 and isbn.isdigit()

        def __str__(self):
            return f"{self.title} by {self.author} - {self.status.name}"

    _instance = None # Singleton

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.books = []
        return cls._instance

    def add_book(self, title, author, isbn):
        if Library.Book.validate_isbn(isbn):
            book = Library.Book(title, author, isbn)
            self.books.append(book)
```

```

        return book
    else:
        raise ValueError("Invalid ISBN")

    def checkout_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn and book.status == Library.BookStatus.AVAILABLE:
                book.status = Library.BookStatus.CHECKED_OUT
                return True
        return False

    def display_books(self):
        print(f"Total books in system: {Library.Book.get_book_count()}")
        for book in self.books:
            print(f"    {book}")

# Use the library (Singleton ensures one library)
lib1 = Library()
lib1.add_book("Python Programming", "John Smith", "1234567890123")
lib1.add_book("Data Science", "Jane Doe", "9876543210987")

lib2 = Library() # Gets same library instance
print(lib1 is lib2) # True

lib2.checkout_book("1234567890123")
lib1.display_books() # Shows book as checked out

```

8. Best Practices and When to Use Each

Class Variables

Use when:

- Data shared by all instances
- Constants that apply to the class
- Counters or shared state

```

class Product:
    tax_rate = 0.07 # Shared by all products

    def __init__(self, price):
        self.price = price

```

Static Methods

Use when:

- Utility function related to class
- Doesn't need instance or class data
- Pure function that fits conceptually with the class

```
class StringHelper:
    @staticmethod
    def reverse(text):
        return text[::-1]
```

Class Methods

Use when:

- Alternative constructors
- Need to access/modify class variables
- Factory methods

```
class Date:
    @classmethod
    def from_string(cls, date_str):
        # Parse and return new instance
        pass
```

Enums

Use when:

- Fixed set of related constants
- Want to prevent invalid values
- Need named constants

```
class Priority(Enum):
    LOW = 1
    MEDIUM = 2
    HIGH = 3
```

Singleton

Use when:

- Exactly one instance needed

- Global access point required
- Resource management (database, config)

Use sparingly - can make testing difficult

Nested Classes

Use when:

- Helper class only used by outer class
- Logical grouping of related classes
- Implementation detail of outer class

```
class Tree:
    class Node: # Only used within Tree
        pass
```

Key Takeaways

- ✓ **Class variables** are shared by all instances, instance variables are unique to each
- ✓ **Static methods** (`@staticmethod`) don't access instance or class data
- ✓ **Class methods** (`@classmethod`) receive the class as first argument (`cls`)
- ✓ **Enums** define sets of named constants (use `from enum import Enum`)
- ✓ **Singleton pattern** ensures only one instance exists (control `__new__`)
- ✓ `__new__` creates the object before `__init__` initializes it
- ✓ **Nested classes** group related classes together
- ✓ **Class methods** are great for alternative constructors
- ✓ **Enums** prevent invalid values and make code more readable
- ✓ **Singleton** is powerful but can complicate testing

Reflection Questions

1. What's the difference between a class variable and an instance variable?
2. When would you use a static method vs a class method?
3. What problem does the Singleton pattern solve?
4. Why use enums instead of strings or numbers?

5. What's the difference between `__new__` and `__init__`?
 6. When is a nested class appropriate?
 7. How do you access a class variable from a class method?
 8. What happens if you assign to a class variable through an instance?
-

Practice Exercises

1. **Counter Class:** Create a class that tracks total instances created using a class variable
2. **Temperature Converter:** Create a class with static methods for celsius/fahrenheit/kelvin conversions
3. **Person Factory:** Create a Person class with class methods to create from different data formats
4. **Traffic Light:** Use enums for RED, YELLOW, GREEN and create a TrafficLight class
5. **Singleton Logger:** Implement a Logger singleton that writes to a file
6. **Nested Car/Engine:** Create Car class with nested Engine class
7. **Order Status:** Use enum for order status (PENDING, SHIPPED, etc.) in an Order class
8. **Database Manager:** Create singleton DatabaseManager with class method to get instance
9. **Priority Queue:** Use enum for priorities (LOW, MEDIUM, HIGH) in a Task class
10. **Complex Example:** Create a Company class with nested Employee class, employee count as class variable, and factory class methods